

المرجع الشامل للبرمجة كائنية التوجه (OOP) في بايثون: من الفكرة إلى الإتقان

مقدمة: لماذا نحتاج إلى OOP أصلاً؟

تخيل أنك تبني كوخًا صغيرًا. يمكنك استخدام أدوات بسيطة وقطع خشب متناثرة لإنجاز المهمة. لكن ماذا لو طلب منك بناء مدينة كاملة؟ الطريقة العشوائية لن تنجح. ستحتاج إلى "مخططات بناء" لكل نوع من المباني (منازل، مستشفيات، مدارس)، وستحتاج إلى طريقة لتنظيم العلاقات بين هذه المباني.

هذا هو بالضبط ما تفعله البرمجة كائنية التوجه (OOP). هي ليست لغة برمجة، بل هي طريقة تفكير وأسلوب لتنظيم الكود يسمح لنا ببناء برامج كبيرة ومعقدة عن طريق تجميع الأكواد في "كائنات" منظمة، تمامًا مثل مباني المدينة.

الفصل الأول: مخطط البناء والمنتج الفعلي (Classes & Objects)

1.1 الكلاس class والكائن object

- **الكلاس (Class):** هو مخطط البناء. إنه يصف الخصائص (attributes) والسلوكيات (methods) التي سيمتلكها أي شيء يتم بناؤه منه. هو مجرد قالب نظري.
- **الكائن (Object/Instance):** هو المنتج الفعلي الذي يتم بناؤه من مخطط البناء. كل كائن هو نسخة مستقلة لها بياناتها الخاصة.

Python

```
class # هو مخطط البناء للموظفين
class Employee:
    pass
```

```
emp1 # و emp2 هما كائنان فعليان تم بناؤهما من المخطط
emp1 = Employee()
emp2 = Employee()
```

1.2 المصنع __init__ وضمير المتكلم self

- **__init__:** هي دالة "المصنع" أو "خط التجميع". يتم استدعاؤها تلقائيًا في كل مرة ننشئ فيها كائنًا جديدًا. وظيفتها هي إعداد الكائن وتعيين خصائصه الأولية.
- **self:** هو ضمير المتكلم الذي يشير إلى "الكائن الذي يتم بناؤه حاليًا". عندما تكتب `self.name = name`، فأنت تقول: "في هذا الكائن المحدد الذي أبينه الآن، اجعل خاصية الاسم تساوي القيمة التي استلمتها".

1.3 الخصائص (Attributes) والسلوكيات (Methods)

- **Attributes:** هي البيانات أو المتغيرات التي تصف حالة الكائن (مثل `name, price, color`).
- **Methods:** هي الدوال الموجودة داخل الكلاس والتي تصف الأفعال التي يمكن للكائن القيام بها.

Python

```
class Employee:
```

```

def __init__(self, fName, lName, salary):
    # Attributes
    self.fName = fName
    self.lName = lName
    self.salary = salary

    # Method
    def fullname(self):
        return "{} {}".format(self.fName, self.lName)

emp1 = Employee("Yousef", "Ahmed", 5000)
print(emp1.fullname()) # استدعاء دالة (سلوك)

```

تمرين صغير: أنشئ كلاسًا بسيطًا Student له خصائص name و student_id، ودالة display_info تطبع هذه المعلومات.

الفصل الثاني: البيانات المشتركة والأعمدة الأربعة

2.1 متغيرات الكلاس (Class Attributes)

هي خصائص مشتركة تنتمي إلى "مخطط البناء" نفسه وتشاركها جميع الكائنات. إذا تغيرت، فإنها تتغير للجميع. فكر فيها كـ "مذكرة داخلية" تُعمم على كل موظفي الشركة.

Python

```

class Employee:
    raise_amount = 1.04 # متغير كلاس

    def apply_raise(self):
        self.salary = int(self.salary * self.raise_amount) # يمكن الوصول له عبر self

```

2.2 الأعمدة الأربعة للـ OOP

1. **التغليف (Encapsulation):** مبدأ تجميع البيانات والسلوكيات في وحدة واحدة (الكلاس). آلية تنفيذه هي عبر جعل المتغيرات "خاصة" والتحكم في الوصول إليها.
 - **عام (Public):** مثل self.name، يمكن الوصول له مباشرة.
 - **محمي (Protected):** يبدأ بـ _ مثل self._name. هذا "اتفاق شرف" بين المبرمجين على عدم استخدامه خارج الكلاس.
 - **خاص (Private):** يبدأ بـ __ مثل self.__name. بايثون تقوم بتغيير اسمه داخليًا (Name Mangling) ليصعب الوصول إليه، وهو يستخدم لمنع التضارب في الوراثة.
2. **التجريد (Abstraction):** إخفاء التفاصيل المعقدة خلف واجهة بسيطة. أنت تستخدم emp1.apply_raise() دون الحاجة لمعرفة كيف تتم العملية الحسابية بالضبط.
3. **الوراثة (Inheritance):** قدرة كلاس جديد (ابن) على أن يرث من كلاس موجود (أب).
4. **تعددية الأشكال (Polymorphism):** قدرة كائنات مختلفة على الاستجابة لنفس استدعاء الدالة كل بطريقته الخاصة.

تمرين صغير: أنشئ كلاس Animal ودالته make_sound. ثم أنشئ كلاسين ابن Dog و Cat يرثان منه ويعيدان تعريف make_sound ليصدرا صوتهما الخاص.

الفصل الثالث: الوراثة المتقدمة والأدوات الخاصة

3.1 super () : لتواصل مع الأب

هي أداة ذكية تسمح للابن باستدعاء دوال الأب super().__init__() . ضرورية لتجنب تكرار الكود، و super().some_method() تسمح لنا بتوسيع سلوك الأب.

Python

```
class Developer(Employee):
    def __init__(self, fName, lName, phone, salary, prog_lang):
        super().__init__(fName, lName, phone, salary) # استدعاء مصنع الأب
        self.prog_lang = prog_lang
    @staticmethod
    @classmethod
```

- **@classmethod:** دالة خاصة بالكلاس (تستقبل cls). أفضل استخدام لها هو كـ "مصنع بديل" لإنشاء كائنات من بيانات غير قياسية.
- **@staticmethod:** دالة مساعدة عامة لا تحتاج للكلاس أو الكائن.

تمرين صغير: أنشئ كلاس Car. اجعل __init__ يستقبل make_model. ثم أنشئ @classmethod اسمه from_string يستقبل نصًا "Toyota-Camry" وينشئ منه كائن سيارة.

الفصل الرابع: جعل الكائنات تتصرف كأنها مدمجة

4.1 __str__ و __repr__

- **__str__:** للطباعة سهلة للقراءة للمستخدم (print(obj)). مثل واجهة السيارة.
- **__repr__:** للطباعة الفنية للمطور، وقاعدتها الذهبية هي أن تُرجع كودًا صالحًا لإنشاء الكائن مرة أخرى. مثل تقرير الميكانيكي.

4.2 Operator Overloading (__add__, __len__)

جعل المعاملات مثل + و len() تعمل مع كائناتك.

Python

```
# داخل كلاس Employee
def __add__(self, other):
    return self.salary + other.salary
```

4.3 الخصائص (Properties)

هي الطريقة الاحترافية للتحكم في الوصول للمتغيرات (Getters/Setters).

- الـ **Getter (@property)**: يتم تنفيذه عند قراءة قيمة المتغير.
- الـ **Setter (*setter)**: يتم تنفيذه عند تعيين قيمة جديدة، ويسمح بالتحقق من صحتها.
- **مشكلة الحلقة اللانهائية**: لتجنبها، نستخدم `_` لاسم المتغير الحقيقي (`self._name`) بينما يبقى اسم الخاصية عامًا (`self.name`).

تمرين صغير: أنشئ كلاس `Product` بخاصية `price` اجعل الـ `Setter` يتأكد من أن السعر لا يمكن أن يكون سالبًا.

Properties , Getter , Setter , Deleter

##مراجعة شاملة: Getters, Setters, Deleters

1. الـ Getter الحصول على القيمة

هذا هو الجزء الذي سألت عنه. الدالة التي تضع فوقها `@property` هي الـ **Getter!** كوري شيفر شرحها لكنه استخدم مصطلح "property" لوصفها.

- ما هو؟ هو الكود الذي يتم تنفيذه تلقائيًا عندما تحاول قراءة قيمة الخاصية.
- الكود:

Python

```
@property
def fullname(self):
    # هذا هو الـ Getter
    print("Getting fullname!")
    return f"{self.fName} {self.lName}"
```

- (الاستدعاء) `print(emp1.fullname)`: ينفذ الكود أعلاه.

2. الـ Setter تعيين قيمة جديدة

- ما هو؟ هو الكود الذي يتم تنفيذه تلقائيًا عندما تحاول تعيين قيمة جديدة للخاصية باستخدام علامة `=`.
- الكود:

Python

```
@fullname.setter
def fullname(self, name):
    # هذا هو الـ Setter
    print("Setting fullname!")
    fName, lName = name.split(' ')
```

```
self.fName = fName
self.lName = lName
```

- (الاستدعاء) emp1.fullname = "Yousef Ahmed": ينفذ الكود أعلاه.
- الفائدة: يسمح لك بالتحقق من صحة القيمة الجديدة أو، كما في هذا المثال، تحديث متغيرين آخرين (self.fName, self.lName) بناءً على قيمة واحدة.

3. Deleter حذف القيمة

- ما هو؟ هو الكود الذي يتم تنفيذه تلقائياً عندما تستخدم كلمة del على الخاصية.
- الكود:

Python

```
@fullname.deleter
def fullname(self):
    Deleter هذا هو #
    print("Deleting fullname!")
    self.fName = None
    self.lName = None
```

- (الاستدعاء) del emp1.fullname: ينفذ الكود أعلاه.

الخلاصة

إدًا، أنت لا تعرف الدالة fullname ثلاث مرات. أنت تعرف ثلاث عمليات منفصلة لخاصية واحدة اسمها fullname:

1. ماذا تفعل عند قراءتها. (Getter)
2. ماذا تفعل عند الكتابة عليها. (Setter)
3. ماذا تفعل عند حذفها. (Deleter)

الفائدة الكبرى من كل هذا هو أنه يمنحك تحكماً كاملاً في المتغيرات، بينما يظل المستخدم النهائي للكلاس يتعامل معها كأنها متغيرات عادية وبسيطة. (emp1.fullname) هذا مثال رائع على التجريد. (Abstraction)

Special Methods الفصل الخامس :

ما هي الـ Special Methods ؟

ببساطة، هي مجموعة من الدوال المعرفة مسبقاً في بايثون والتي تبدأ وتنتهي بشرطتين مزدوجتين (مثلاً `__init__`). هذه الدوال تسمح للكائنات التي ننشئها بأن تتفاعل مع العمليات المدمجة في اللغة، مثل الجمع (+) ، الطرح (-) ، دالة `len()` ، والطباعة `print()`.

الأداة الذهنية :فكر في هذه الدوال كأنها "أزرار" خاصة في الكائن الخاص بك. بايثون تعرف هذه الأزرار، وعندما تقوم بعملية معينة، تبحث بايثون عن الزر المناسب في الكائن الخاص بك وتضغط عليه.

أهم الدوال الخاصة التي تمت مراجعتها

1. `__init__(self, ...)`: المصنع

هذه هي أشهر دالة خاصة، وقد أتقنتها بالفعل. هي "الزر" الذي تضغطه بايثون تلقائياً عند إنشاء كائن جديد.

2. `__repr__(self)` و `__str__(self)`: بطاقة التعريف

هاتان الدالتان مسؤولتان عن تحويل الكائن الخاص بك إلى نص (string) ، لكن لكل منهما هدف مختلف.

- **`__repr__(self)` (Representation):**
 - الهدف :إرجاع نص واضح وغير غامض يصف الكائن، وهو موجه للمبرمجين. القاعدة العامة هي أن هذا النص يجب أن يكون، إن أمكن، كود بايثون صالحاً يمكنه إعادة إنشاء الكائن.
 - الأداة الذهنية :فكر فيه كأنه "الرقم التسلسلي" أو "بطاقة المواصفات الفنية" للمنتج.
 - مثال `: return f"Employee('{self.first}', '{self.last}', {self.pay})"`
- **`__str__(self)` (String):**
 - الهدف :إرجاع نص سهل القراءة وجميل، وهو موجه للمستخدم النهائي. هذه هي الدالة التي تستدعيها `print()` تلقائياً.
 - الأداة الذهنية :فكر فيه كأنه "اسم المنتج ووصفه" على العبلة.
 - مثال `: return f'{self.fullname()} - {self.email}'`
- ملاحظة :إذا لم تقم بتعريف `__str__` ، فإن `print()` ستبحث عن `__repr__` وتستخدمها بدلاً منها.

Operator Overloading: جعل الكائنات تتصرف كأنها مدمجة

هنا يكمن قلب الموضوع الذي سألك عنه الدكتور.

1. `__add__(self, other)`: التحميل الزائد+

- **الهدف:** تعريف ما يحدث عند استخدام علامة الجمع (+) بين كائنات من الكلاس الخاص بك.
- **مثال كوري شيفر:** قام بتعريفها لجمع رواتب الموظفين.

Python

```
def __add__(self, other):
    return self.pay + other.pay
```

- **الاستخدام** `print(emp1 + emp2)`: سيقوم الآن بجمع راتب الموظف الأول مع الثاني.

2. التحميل الزائد `len()` `__len__(self):`

- **الهدف:** تعريف ما يحدث عندما تستدعي دالة `len()` المدمجة على الكائن الخاص بك.
- **مثال كوري شيفر:** قام بتعريفها لترجع طول الاسم الكامل للموظف.

Python

```
def __len__(self):
    return len(self.fullname())
```

- **الاستخدام** `print(len(emp1))`: سيعيد الآن عدد حروف الاسم الكامل للموظف.

الخلاصة :

الدوال الخاصة هي بوابتك لجعل الكلاسات التي تنشئها تتكامل بسلاسة مع لغة بايثون، وتتصرف تمامًا مثل الأنواع المدمجة (strings, lists, integers)، مما يجعل الكود الخاص بك أكثر وضوحًا وقوة.

الفصل السادس: الغوص العميق في آليات ومبادئ OOP

1. التغليف (Encapsulation): بناء الحصن المنيع

- المبدأ: هو تجميع البيانات (Attributes) والعمليات التي تجري عليها (Methods) في وحدة واحدة (الكلاس)، والأهم من ذلك، التحكم في الوصول إلى هذه البيانات. الهدف هو حماية بيانات الكائن من التعديل العشوائي وغير المنطقي.
- آليات التنفيذ في بايثون:
 - العام (Public): أي متغير لا يبدأ بـ _ مثل (self.name). يمكن الوصول إليه وتعديله من أي مكان. مثل واجهة المحل.
 - المحمي (Protected): يبدأ بشرطة سفلية واحدة (_ مثل (self._balance). هذا لا يمنع الوصول، بل هو "اتفاق شرف" بين المبرمجين يقول: "هذا متغير داخلي، لا يفترض بك تعديله مباشرة من خارج الكلاس". مثل غرفة "الموظفين فقط".
 - الخاص (Private): يبدأ بشرطتين سفليتين (__ مثل (self.__id). بايثون تقوم بإعادة تسميته داخليًا (_ClassName__id) ليصعب جدًا الوصول إليه. يستخدم لمنع التضارب العرضي في الوراثة. مثل خزانة البنك).

2. التجريد (Abstraction): قيادة السيارة دون فهم المحرك

- المبدأ: إخفاء التفاصيل التنفيذية المعقدة خلف واجهة بسيطة وسهلة الاستخدام. الهدف هو تقليل التعقيد للمستخدم الذي يتعامل مع الكائن.
- مثال عملي: عندما أنشأنا كلاس Inventory، المستخدم النهائي يتعامل مع دالة بسيطة جدًا اسمها inventory.total_inventory_value. هو لا يحتاج أن يعرف أن هذه الدالة في الخلفية تقوم بالمرور على قاموس، وتصل إلى كل كائن، وتستدعي خاصية total_value لكل كائن، وتجمع النتائج. كل هذا التعقيد تم "تجريده" خلف واجهة بسيطة.

3. إعادة التعريف (Overriding): بصمة الابن الخاصة

- المبدأ: هو أن يقوم الكلاس الابن بتقديم تنفيذ خاص به لدالة ورثها من الأب. هذا يسمح للابن بتخصيص السلوك الموروث.
- أنواعه:
 - الاستبدال الكامل: الابن يتجاهل تمامًا ما يفعله الأب. (مثال: دالة make_sound في Lion استبدلت بالكامل دالة Animal).
 - التوسيع والإضافة: الابن ينفذ منطق الأب أولاً باستخدام super().method()، ثم يضيف عليه منطقاً الخاص. (مثال: دالة info في ElitePlayer استدعت super().info() ثم أضافت معلومات العنصر الخاص).

4. تعددية الأشكال (Polymorphism): نفس الزر، نتائج مختلفة

- المبدأ : هو قدرة كائنات من كلاسات مختلفة على الاستجابة لنفس استدعاء الدالة، كل بطريقته الخاصة.
- كيف يعمل في بايثون (Duck Typing) : بايثون لا يهتم بنوع الكائن بقدر ما يهتم بما إذا كان الكائن يستطيع تنفيذ الدالة المطلوبة. في مثال حديقة الحيوان، عندما مررنا على قائمة zoo_animals واستدعينا animal.make_sound()، بايثون لم تقل "هل هذا أسد؟"، بل قالت "هل هذا الشيء animal لديه دالة make_sound؟ إذا نعم، نفذها". هذا المبدأ يجعل الكود مرناً جداً.

5. الدوال الخاصة (Special Methods) وتحميل المعاملات (Operator Overloading)

هذان المصطلحان مرتبطان بشكل وثيق. الدوال الخاصة هي الآلية التي تنفذ بها بايثون تحميل المعاملات.

- (Overloading التحميل الزائد): (هو جعل "المعامل (Operator) "مثل + أو > أو len() يفهم كيفية التعامل مع الكائنات المخصصة التي ننشئها.
- كيف؟ عن طريق تعريف الدالة الخاصة المقابلة له:
 - لجعل + يعمل، نعرّف __add__.
 - لجعل == يعمل، نعرّف __eq__.
 - لجعل len() يعمل، نعرّف __len__.
 - لجعل obj[key] يعمل، نعرّف __getitem__.
- المصطلح: overloading أحياناً يستخدم في لغات أخرى (مثل Java/C++) للإشارة إلى وجود دالتين بنفس الاسم لكن بمعاملات مختلفة. بايثون لا تدعم هذا بشكل مباشر، والحل البديل هو استخدام المعاملات الاختيارية. ما نقصده هنا هو Operator Overloading.

6. Deleters: منطق الحذف المخصص

- المبدأ : الـ Deleter (@property.deleter) هو الجزء الثالث من الخاصية (Property)، ويتم تنفيذه عند استخدام del على الخاصية.
- الغاية : وظيفته ليست مجرد حذف المتغير، بل تنفيذ منطق مخصص عند الحذف. في مثال الحساب البنكي، لم نقوم فقط بحذف overdraft_limit، بل قمنا بـ:
 1. التحقق من شرط: هل الرصيد سالب؟ (منع الحذف).
 2. تنفيذ إجراء: إعادة تعيين القيمة إلى 0.
- إنه يمنحك القدرة على التحكم في دورة حياة الخاصية بشكل كامل.

الفصل السابع: دفتر أخطائي (الدروس المستفادة)

- الخطأ: الشرط يعمل لكن الخطأ لا يحدث.
 - الدرس: يجب استخدام كلمة `raise Exception` ("إطلاق الخطأ").
- الخطأ: `: TypeError: 'str' object is not callable`.
 - الدرس `@property`: تحول الدالة إلى خاصية، لذا يجب إزالة الأقواس () عند الوصول إليها.
- الخطأ: `: AttributeError: 'function' object has no attribute 'setter'`.
 - الدرس: يجب تعريف الـ `Getter` قبل الـ `Setter`.
- الخطأ: الـ `\n` الخفي عند قراءة الملفات.
 - الدرس: دائماً استخدم `.strip()` على السطور المقروءة لتنظيفها.

الفصل الثامن : ما هي الوراثة متعددة المستويات؟

هي ببساطة سلسلة من الوراثة، مثل شجرة العائلة.

- **الجد (Device):** لديه خصائص عامة جدًا.
- **الأب (ElectricDevice):** يرث من الجد، ويضيف خصائصه الخاصة.
- **الابن (PhoneDevice):** يرث من الأب. وبما أن الأب ورث من الجد، فإن الابن يرث بشكل غير مباشر من الجد أيضًا.

مثال عملي خطوة بخطوة

دعنا نبني المثال الذي ذكرته:

1. **الجد (Device):** الجهاز (أي جهاز في العالم له رقم تعريف).

```
Python
class Device:
    def __init__(self, device_id):
        self.device_id = device_id
    print(f"Device with ID {self.device_id} created.")

    def get_id(self):
        return f"My ID is {self.device_id}"
```

2. **الأب (ElectricDevice):** الجهاز الكهربائي (هو "جهاز"، لكنه يضيف شيئًا جديدًا: يحتاج إلى كهرباء).

```
Python
# يرث من Device
class ElectricDevice(Device):
    def __init__(self, device_id, power_rating):
        # نستدعي مصنع "الجهاز" لنحصل على خاصية id
        super().__init__(device_id)
        self.power_rating = power_rating
    print("I am an electric device.")

    def plug_in(self):
        print("Plugging into power source...")
```

3. **الابن (PhoneDevice):** الهاتف (هو "جهاز كهربائي"، لكنه يضيف شيئًا جديدًا: يمكنه إجراء المكالمات).

```
Python
# يرث من ElectricDevice
class PhoneDevice(ElectricDevice):
    def __init__(self, device_id, power_rating, phone_number):
        # نستدعي مصنع "الجهاز الكهربائي"
        # وهذا بدوره سيستدعي مصنع "الجهاز"
        super().__init__(device_id, power_rating)
        self.phone_number = phone_number
    print("I am a phone.")
```

```
def make_call(self):  
    print(f"Calling from {self.phone_number}...")
```

ماذا يمتلك الابن (PhoneDevice) الآن؟

الآن، لننشئ كائنًا من الابن ونرى ما يمكنه فعله:

Python

```
my_phone = PhoneDevice("SN123", "5V", "555-1234")
```

```
# 1. دالة من نفسه مباشرة  
my_phone.make_call()
```

```
# 2. دالة موروثية من أبيه المباشر (ElectricDevice)  
my_phone.plug_in()
```

```
# 3. دالة موروثية من جده (Device)  
print(my_phone.get_id())
```

النتائج المتوقعة:

```
Device with ID SN123 created.  
I am an electric device.  
I am a phone.  
Calling from 555-1234...  
Plugging into power source...  
My ID is SN123
```

كما ترى، الابن (PhoneDevice) ورث كل شيء من سلسلة أسلافه. دالة `super()` دائمًا ما تشير إلى الأب المباشر، وهذا ما يضمن أن سلسلة البناء (`__init__`) تعمل بشكل صحيح من الجد إلى الأب ثم إلى الابن.

الفصل التاسع : بنك التمارين الشامل 🏆

تمارين سهلة (Easy Level)

الهدف : التأكد من إتقان بناء الكلاسات والمفاهيم الأساسية.

1. الكوكب :أنشئ كلاس Planet يستقبل name و mass في __init__.
2. أغنيتي المفضلة :أنشئ كلاس Song يستقبل title و artist، ودالة play تطبع {title} "Now playing by {artist}".
3. مستطيل بسيط :أنشئ كلاس Rectangle يستقبل width و height، ودالة calculate_area تُرجع المساحة.
4. مستخدم الموقع :أنشئ كلاس User له متغير كلاس MIN_PASSWORD_LENGTH = 8.
5. بطاقة شخصية :أنشئ كلاس Person يستقبل name و age، وقم بتعريف __str__ لطباعة "Person: {name}, Age: {age}".
6. حيوان بسيط :أنشئ كلاس أب Animal وكلاس ابن Dog يرث منه ويضيف خاصية breed.
7. سيارة كهربائية :أنشئ كلاس أب Car وكلاس ابن ElectricCar يرث منه ويعيد تعريف متغير fuel_type من "Gasoline" إلى "Electric".
8. جمع الأعداد :أنشئ كلاس Number يستقبل value، وقم بتعريف __add__ ليجمع قيمة كائنين من Number.
9. طول الكلمة :أنشئ كلاس Word يستقبل text، وقم بتعريف __len__ ليعيد طول النص.
10. صانع النقاط :أنشئ كلاس Point يستقبل x و y. أنشئ @classmethod اسمه from_tuple يستقبل (x, y) وينشئ منه كائن Point.

تمارين متوسطة (Medium Level)

الهدف :دمج مفهومي أو ثلاثة في كل تمرين.

1. الموظف والمدير :أنشئ كلاس أب Employee وكلاس ابن Manager يرث منه Manager. يجب أن يكون لديه قائمة employees خاصة به ودوال لإضافة وإزالة الموظفين.
2. الكتاب والمكتبة :أنشئ كلاس Book وكلاس Library. Library يحتوي على قائمة من كائنات Book وله دالة __len__ لتعيد عدد الكتب.
3. السيارة من نص :أنشئ كلاس Car ودالة @classmethod اسمها from_string تستقبل "Make-Model-Year" وتنشئ كائناً منه مع تحويل السنة إلى رقم int.
4. درجة الحرارة الآمنة :أنشئ كلاس Temperature بخاصية celsius. الـ Setter الخاص بها يجب أن يمنع تعيين درجة حرارة أقل من -273.15.
5. حقيقة الحيوان :أنشئ كلاس Animal وأبناء Fish و Bird. اكتب دالة خارجية تستقبل قائمة حيوانات وتستخدم isinstance لتجعل السمكة تسبح والطائر يطير.
6. المتجر الإلكتروني :أنشئ كلاس Product بخاصية price (يجب أن تكون موجبة) وخاصية quantity (يجب أن تكون موجبة أو صفر).
7. متجهات رياضية :أنشئ كلاس Vector يدعم الجمع (__add__) والمقارنة (__eq__) و __lt__ بناءً على طول المتجه.
8. صندوق بريد :أنشئ كلاس Mailbox يحتوي على قائمة رسائل. قم بتعريف __getitem__ للوصول لرسالة معينة عبر my_mailbox[i] و __str__ لطباعة عدد الرسائل.

9. المربع: أنشئ كلاس Square يرث من Rectangle. يجب أن يكون `__init__` الخاص به يستقبل ضلعًا واحدًا فقط ويستخدم `super()` لتمريره للطول والعرض في الأب.
10. التحقق من التاريخ: أنشئ كلاس Date ودالة `@staticmethod` اسمها `is_valid_date` تستقبل يوم وشهر وسنة وتتحقق مما إذا كانت صالحة (بشكل مبسط).

تمارين صعبة (Hard Level)

الهدف: التعامل مع منطق معقد وتفاعل أكبر بين مكونات الكلاس.

- النظام البنكي: تمرين 3 الأصلي `BankAccount -> CheckingAccount` (مع رسوم سحب. الابن يعيد تعريف `withdraw` ليضيف الرسوم قبل استدعاء `super()`).
- أسطول الشاحنات: تمرين 4 الأصلي `Vehicle -> Truck` (تعامل مع وراثة متغيرات الكلاس وإعادة تعريف `@classmethod`).
- المستطيل الهندسي: تمرين متقدم 3 `Rectangle` (مع `Getters/Setters` للتحقق، خاصية `area` للقراءة فقط، `__eq__` للمقارنة بالمساحة، و `@classmethod` لإنشاء مربع).
- الحساب البنكي المتقدم: تمرين متقدم 4 `PremiumAccount` (مع خاصية `overdraft_limit` التي تستخدم `Getter, Setter` و `Deleter` بمنطق مخصص).
- الأبطال الخارقين: تمرين 5 الأصلي `SuperHero` (مع `FlightEnabled` و `TechPowered` ك `Mixins` لتطبيق الوراثة المتعددة).
- كلاس متعدد الأشكال: أنشئ كلاس أب `Document` له دالة `render`. أنشئ أبناء `PdfDocument`, `WordDocument`, `HtmlDocument` كل منهم يطبق `render` بطريقته. اكتب دالة `render_all(docs)` تطبع كل أنواع المستندات.
- عداد مخصص: أنشئ كلاس `Counter` يمكن جمعه وطرحه من كائنات `Counter` أخرى (`__add__`, `__sub__`) ويمكن مقارنته (`__gt__`, `__lt__`, `__eq__`).
- بطاقات اللعب: أنشئ كلاس `Card` (له قيمة وشكل) وكلاس `Deck` يحتوي على 52 كائن `Card`. يجب أن يكون له `__len__` ودالة `shuffle` لخلط البطاقات ودالة `deal` لسحب بطاقة.
- التحقق من الصحة المتقدم: تمرين متقدم 3 `DataValidator` (مع دوال `@staticmethod` للتحقق من الإيميل وكلمة المرور، ودالة `@classmethod` تقرأ من ملف وتطبق هذا التحقق).
- درجة الحرارة النكية: تمرين متقدم 1 `Temperature` (مع خاصيتي `celsius` و `fahrenheit` حيث الـ `Setter` الخاص بكل منهما يؤثر على الآخر).

تمارين متقدمة (Advanced Level)

الهدف: بناء أنظمة صغيرة و مترابطة، والتركيز على التصميم.

- صلاحيات المستخدمين: تمرين متقدم 2 `WebsiteUser -> Moderator` (إعادة تعريف الـ `Properties` والـ `Setters` مع استدعاء منطق الأب).
- نظام إدارة المخزون (مصغر): `InventoryItem -> SellableItem`: كلاس `Inventory` يحتوي على قاموس من هذه الكائنات وخاصية `total_inventory_value` تحسب القيمة الإجمالية.
- كلاس `Matrix`: أنشئ كلاسًا لتمثيل مصفوفة رياضية (قائمة من القوائم). قم بتعريف `__str__` لطباعتها بشكل جميل، و `__add__` لجمع مصفوفتين، و `__mul__` لضرب مصفوفتين.

4. **كلاس متعدد الحدود (Polynomial):** أنشئ كلاسًا يمثل دالة رياضية متعددة الحدود (مثل $3x^2 + 5x - 2$). قم بتعريف `__init__` ليستقبل المعاملات، `__str__` لطابعاتها، و `__call__` لتقييم الدالة عند قيمة x معينة.
5. **محفظة أسهم:** أنشئ كلاس `Stock` (له رمز وسعر) وكلاس `Portfolio`. `Portfolio` يحتوي على قاموس من الأسهم وكمياتها. يجب أن يكون له خاصية `total_value` تحسب القيمة الإجمالية للمحفظة.
6. **كلاس Fraction:** أنشئ كلاسًا لتمثيل الكسور (بسط ومقام). يجب أن يقوم بتبسيط الكسر تلقائيًا عند إنشائه. قم بتعريف العمليات الحسابية (`__add__`, `__sub__`, `__mul__`, `__truediv__`).
7. **نظام حجز مواعيد:** أنشئ كلاس `Appointment` وكلاس `Calendar`. `Calendar` يمكنه إضافة موعد والتحقق من عدم وجود تضارب في المواعيد.
8. **محرر نصوص بسيط:** أنشئ كلاس `TextEditor` له دوال `undo` و `redo` و `__str__` لعرض النص الحالي.
9. **آلة الحالة المحدودة (Finite State Machine):** أنشئ كلاسًا يمثل آلة حالة بسيطة (مثل إشارة المرور: أخضر -> أصفر -> أحمر). له دالة `next_state` تنتقله للحالة التالية.
10. **كلاس API Client:** أنشئ كلاسًا يغلف طلبات `API`. `__init__` يأخذ الـ `URL` الأساسي. الدوال (مثل `get_users`) تقوم بعمل طلبات `API` وتعيد البيانات بصيغة `JSON`.

تمارين مستوى خبير (Expert Level)

الهدف: تصميم وحل مشاكل تتطلب عدة كلاسات متفاعلة بشكل معقد.

1. **لعبة RPG نصية (مصغرة):** أنشئ كلاسات `Character`, `Weapon`, `Armor`. `Character` يمكنه تجهيز سلاح ودرع (`equip`). دالة `attack` تأخذ `target` (شخصية أخرى) وتحسب الضرر بناءً على قوة سلاح المهاجم ودفاع درع المدافع.
2. **نظام ملفات وهمي:** أنشئ كلاس أب `FileSystemNode` وكلاسات ابن `File` و `Directory`. `Directory` يمكن أن يحتوي على قائمة من `FileSystemNode` أخرى. قم بتعريف دوال مثل `search` للبحث عن ملف.
3. **مكتبة شخصية:** أنشئ كلاس `Book` وكلاس `Library`. `Library` يمكنه إضافة وحذف الكتب، والبحث عن كتاب بالاسم أو المؤلف، وإعارة الكتب وتتبع من استعارها وتاريخ الإرجاع.
4. **محلل بيانات CSV:** أنشئ كلاس `CSVParser` يقرأ ملف `CSV`. يجب أن يكتشف تلقائيًا أسماء الأعمدة وأنواع البيانات في كل عمود (`int`, `float`, `str`). يجب أن يكون له دالة `get_row_as_dict`.
5. **نظام بنكي (متعدد المستخدمين):** أنشئ كلاسات `Bank`, `Account`, `Customer`. `Bank` يمكنه إنشاء عملاء جدد وفتح حسابات لهم `Customer`. يمكنه امتلاك عدة حسابات. قم بتنفيذ دالة `transfer` بين حسابات مختلفة حتى لو كانت لعملاء مختلفين.
6. **محاكي نظام بيئي:** أنشئ كلاس أب `Organism` وكلاسات ابن `Plant` (ينمو) و `Animal` (يتحرك ويأكل). `Animal` ينقسم إلى `Herbivore` (يأكل النباتات) و `Carnivore` (يأكل الحيوانات الأخرى). قم بإنشاء عالم (قائمة ثنائية الأبعاد) وضع فيه هذه الكائنات ودعها تتفاعل.
7. **أداة Web Scraper:** أنشئ كلاس `Scraper` يستقبل `URL` ويستخدم مكتبات مثل `requests` و `BeautifulSoup` لسحب البيانات. أنشئ دوال متخصصة مثل `get_all_links` أو `get_articles`.
8. **إطار عمل ويب (مصغر جدًا):** أنشئ كلاس `App` يمكنه تعريف مسارات (`routes`) مثل `@app.route("/home")`. عندما يتم تشغيله، يستمع لطلبات `HTTP` بسيطة ويرد عليها.
9. **بلوكتشين (مصغر):** أنشئ كلاس `Block` (له بيانات وهاش وهاش البلوك السابق) وكلاس `Blockchain`. `Blockchain` يحتوي على سلسلة من البلوكات ويمكنه إضافة بلوك جديد مع التأكد من صحة السلسلة.
10. **مكتبة رسم بياني:** أنشئ كلاس `Node` وكلاس `Graph`. `Graph` يمكنه إضافة وحذف الـ `Nodes` والـ `Edges` (الروابط) بينها. قم بتنفيذ خوارزميات بسيطة مثل البحث بالعمق (`DFS`) أو البحث بالعرض (`BFS`).

مشاريع متوسطة الحجم

1. إعادة هيكلة تطبيق **ToDo** (المشروع الذي خططنا له) حول تطبيقك الحالي إلى تطبيق OOP متكامل بكلاسات Task و TaskManager.
2. آلة بيع رقمية: أنشئ كلاس Product وكلاس VendingMachine. الآلة يجب أن تدير مخزونًا من المنتجات، تقبل "النقود"، توزع المنتج، وتعيد الباقي. يجب أن تتعامل مع حالات مثل نفاذ المنتج أو عدم كفاية النقود.
3. نظام حجز فندق بسيط: أنشئ كلاسات Room (لها رقم، نوع، سعر، وحالة "محجوز") وكلاس Hotel. Hotel. مدير قائمة من الغرف، يمكنه البحث عن غرفة متاحة، وحجز غرفة لعميل في تواريخ معينة، وإلغاء الحجز.

مشاريع كبيرة وصعبة جدًا

1. نظام إدارة المخزون الكامل: قم بتوسيع تمرين الخبراء. أضف أنواعًا مختلفة من المنتجات (PerishableItem) له تاريخ انتهاء صلاحية). قم بتنفيذ حفظ وتحميل المخزون من وإلى ملفات CSV أو JSON. أضف القدرة على إنشاء تقارير (مثل المنتجات التي قاربت على النفاذ).
2. نظام إدارة الجامعة: قم بتوسيع تمرين الطلاب. أنشئ كلاسات Student, Course, Professor, Department. Department يقدم مسابقات Professor. يدرس مسابقات Student. يسجل في مسابقات ويحصل على درجات. النظام يجب أن يكون قادرًا على حساب معدل الطالب، عرض جدول الأستاذ، وطباعة المسابقات المتاحة في كل قسم.
3. لعبة مغامرات نصية: مشروع OOP كلاسيكي (أنشئ كلاسات Room لها وصف ومخرج)، Player (له مخزون وصحة)، و Item. اللاعب يمكنه التنقل بين الغرف (go north)، النقاط وترك الأشياء (take key)، drop sword، والتفاعل مع البيئة (unlock door).
4. منصة تجارة إلكترونية (الخلفية): أنشئ كلاسات Customer, Product, ShoppingCart, Order. Customer يمكنه إضافة منتجات إلى ShoppingCart الخاص به. عند الدفع، يتم إنشاء كائن Order من محتويات العربة وتفرغها.
5. نظام حجز طيران: أنشئ كلاسات Flight له رقم، مسار، سعة مقاعد، وقائمة ركاب (Passenger، و Booking. النظام يجب أن يسمح بحجز راكب على رحلة، إلغاء الحجز، والتأكد من عدم حجز مقاعد أكثر من سعة الطائرة. يجب أن يكون قادرًا على طباعة قائمة الركاب لكل رحلة.

الفصل العاشر: مصادر ومراجع إضافية للتعمق

هذا الفصل هو بوابتك لتوسيع معرفتك بالبرمجة كائنية التوجه وما بعدها. التعلم لا يتوقف أبدًا، والاطلاع على أساليب شرح مختلفة من مصادر متنوعة هو ما يصنع المبرمج المحترف.

1. ##### كورسات الفيديو (Video Courses)

- **Python OOP Tutorial for Beginners - Corey Schafer:**
 - الرابط: [Corey Schafer's OOP Playlist](#)
 - الوصف: المصدر الذي بدأنا به رحلتنا. يتميز بالعمق والمنهجية، ويركز على فهم "لماذا" تعمل الأشياء بهذه الطريقة. مثالي لبناء فهم صلب.
- **Python OOP Full Course - freeCodeCamp:**
 - الرابط: [freeCodeCamp's OOP Course](#)
 - الوصف: كورس شامل ومكثف مبني على مشروع عملي. رائع لربط كل المفاهيم معًا ورؤيتها في سياق تطبيق واحد متكامل.
- **Python Object Oriented Programming - Tech With Tim:**
 - الرابط: [Tech With Tim's OOP Playlist](#)
 - الوصف: سلسلة أخرى ممتازة تتميز بأسلوب شرح واضح ومباشر، مع الكثير من الأمثلة العملية والتمارين.

2. ##### التوثيق الرسمي لبايثون (Official Python Documentation)

هذا هو المصدر الأم والحقيقة المطلقة لكل ما يخص بايثون. قد يكون صعب القراءة في البداية، لكن التعود عليه مهارة لا تقدر بثمن.

- **The Python Tutorial - Chapter 9: Classes:**
 - الرابط: [Official Python Tutorial on Classes](#)
 - الوصف: الشرح الرسمي من صانعي اللغة لكيفية عمل الكلاسات والكائنات وكل ما يتعلق بها.
- **The Python Data Model (Special Methods):**
 - الرابط: [Official Python Data Model Documentation](#)
 - الوصف: هذا هو القاموس الكامل لكل الدوال الخاصة (Dunder Methods) مثل `__init__`, `__add__`, `__len__` وغيرها. عندما تنسى ماذا تفعل دالة معينة، هذا هو المكان الذي تبحث فيه. مرجع أساسي للمطورين المتقدمين.

3.#### كتب موصى بها(Recommended Books)

عندما تشعر بالاستعداد للانتقال إلى مستوى أعمق، هذه الكتب تعتبر من أفضل ما كُتب عن بايثون والبرمجة كائنية التوجه.

- **"Fluent Python" by Luciano Ramalho:**
 - الوصف: يعتبره الكثيرون أفضل كتاب للانتقال من المستوى المتوسط إلى الاحترافي في بايثون. فصوله عن "نموذج البيانات (Data Model)" و"الدوال الخاصة ستغير طريقة تفكيرك في اللغة". **مستوى متقدم.**
- **"Python Crash Course" by Eric Matthes:**
 - الوصف: واحد من أكثر الكتب مبيعاً وشهرة لتعلم بايثون من الصفر. الجزء الخاص بال-OOP فيه عملي جداً ومبني على مشاريع، مما يجعله مناسباً جداً للمبتدئين.
- **"Head First Python" by Paul Barry:**
 - الوصف: يشتهر بأسلوبه التعليمي الفريد الذي يعتمد على الصور والرسوميات وطرق العرض غير التقليدية لتسهيل فهم المفاهيم المعقدة. إذا كنت متعلماً بصرياً، فهذا الكتاب قد يكون الأنسب لك.

BEST WISHES, THANKS,,